Giải thích chi tiết Code AMR - Phần 1: Khởi tạo & Cảm biến

> Tài liệu giải thích từng dòng code theo đúng thứ tự workflow khi robot khởi động và vận hành.

PHASE 1: KHỞI TẠO HỆ THỐNG (`setup()` trong AMR_MAIN.ino)

1.1 Khai báo thư viện & hằng số

```
#include <WiFi.h>         // Thư viện WiFi cho ESP32
#include <esp_now.h>      // Giao thức ESP-NOW (dự phòng)
#include <Arduino.h>      // Lõi Arduino
#include <math.h>         // Hàm toán học (sin, cos, atan2, sqrt)
#include <Wire.h>         // Giao tiếp I2C (cho IMU)
#include <MPU6050.h>      // Thư viện cảm biến MPU6050
```

Thông số vật lý robot:

```
#define WHEEL_DIAMETER  66.0   // Đường kính bánh xe = 66mm
#define WHEEL_BASE      205.0  // Khoảng cách giữa 2 tâm bánh = 205mm
#define ENCODER_PPR     495.0  // Encoder phát 495 xung mỗi vòng quay bánh
```

> Các giá trị này là nền tảng cho MỌI phép tính odometry và PID. Sai số ở đây sẽ ảnh hưởng toàn bộ hệ thống.

Chân điều khiển:

```
#define IN1 19   // Chiều quay Motor trái - chân 1
#define IN2 18   // Chiều quay Motor trái - chân 2
#define IN3 17   // Chiều quay Motor phải - chân 1
#define IN4 16   // Chiều quay Motor phải - chân 2
#define ENA 25   // PWM điều tốc Motor trái
#define ENB 26   // PWM điều tốc Motor phải
#define STBY 15  // Chân Standby driver (HIGH = bật)
```

1.2 Hàm `setup()` — Trình tự khởi tạo

```
void setup() {
  Serial.begin(115200);  // Mở Serial debug tốc độ 115200 baud
```

Bước 1 — Cấu hình chân Motor:

```
  pinMode(ENA, OUTPUT);   // Chân PWM trái → output
  pinMode(IN1, OUTPUT);   // Các chân chiều quay → output
  pinMode(IN2, OUTPUT);
  pinMode(IN3, OUTPUT);
  pinMode(IN4, OUTPUT);
  pinMode(ENB, OUTPUT);   // Chân PWM phải → output
  pinMode(STBY, OUTPUT);
  digitalWrite(STBY, HIGH); // Bật driver motor
```

Bước 2 — Cấu hình chân Encoder:

```
  pinMode(ENCODER_1A, INPUT_PULLUP);  // Encoder trái kênh A
  pinMode(ENCODER_1B, INPUT_PULLUP);  // Encoder trái kênh B
  pinMode(ENCODER_2A, INPUT_PULLUP);  // Encoder phải kênh A
  pinMode(ENCODER_2B, INPUT_PULLUP);  // Encoder phải kênh B
```

Bước 3 — Gắn ngắt Encoder (Interrupt):

```
  // Mỗi khi kênh A có cạnh lên (RISING), gọi hàm ISR tương ứng
  attachInterrupt(digitalPinToInterrupt(ENCODER_1A), encoder1_isr, RISING);
  attachInterrupt(digitalPinToInterrupt(ENCODER_2A), encoder2_isr, RISING);
```

> ISR = Interrupt Service Routine. Chạy tự động ở nền, không cần gọi thủ công.

Bước 4 — Timer Interrupt 20ms:

```
  timer0 = timerBegin(1000000);          // Tạo timer tần số 1MHz
  timerAttachInterrupt(timer0, &timer0_ISR); // Gắn hàm ISR cho timer
  timerAlarm(timer0, 20, true, 0);       // Kích hoạt mỗi 20 tick = 20ms
```

> Cứ mỗi 20ms, biến `odomFlag` được bật → `loop()` biết phải cập nhật odometry.

Bước 5 — Khởi tạo các module:

```
  IMU_init();   // Khởi tạo + calibrate MPU6050
  Odom_init();  // Reset tọa độ odometry về (0,0,0)
```

```
  BT_init();    // Phát Bluetooth Classic "AMR_Robot"
  WiFi_init();  // Phát WiFi AP + mở TCP Server port 8080
```

PHASE 2: CẢM BIẾN — ENCODER (Encoder.ino)

2.1 `encoder1_isr()` — Ngắt Encoder bánh trái

```
void encoder1_isr() {
  int B = digitalRead(ENCODER_1B);  // Đọc trạng thái kênh B
  if (B == LOW) {
    encoder1_value++;   // Kênh B = LOW khi A lên → quay TIẾN → tăng
  } else {
    encoder1_value--;   // Kênh B = HIGH khi A lên → quay LÙI → giảm
  }
}
```

> **Nguyên lý Quadrature Encoder:** Kênh A và B lệch pha 90°. Khi kênh A có cạnh lên (RISING), ta đọc kênh B để xác định chiều quay.

2.2 `encoder2_isr()` — Ngắt Encoder bánh phải

```
void encoder2_isr() {
  int B = digitalRead(ENCODER_2B);
  if (B == LOW) {
    encoder2_value++;  // Tiến
  } else {
    encoder2_value--;  // Lùi
  }
}
```

> Hoạt động giống hệt encoder1 nhưng cho bánh phải.

PHASE 3: CẢM BIẾN — IMU MPU6050 (IMU.ino)

3.1 `IMU_init()` — Khởi tạo & Calibrate

```
void IMU_init() {
  Wire.begin(IMU_SDA, IMU_SCL);  // Mở I2C: SDA=21, SCL=22
```

```
  Wire.setClock(400000);          // Tốc độ I2C = 400kHz (Fast Mode)
  mpu.initialize();               // Khởi tạo MPU6050
```

Kiểm tra kết nối:

```
  if (mpu.testConnection()) {
    // MPU6050 OK — kết nối thành công
  } else {
    // MPU6050 FAIL — lỗi phần cứng/dây nối
  }
```

Cấu hình thang đo:

```
  mpu.setFullScaleGyroRange(MPU6050_GYRO_FS_250);   // Gyro: ±250°/s
  mpu.setFullScaleAccelRange(MPU6050_ACCEL_FS_2);    // Accel: ±2g
```

> Thang ±250°/s → độ nhạy = 131 LSB/(°/s). Con số 131.0 sẽ xuất hiện khi đổi raw → °/s.

Calibrate offset trục Z (bù trừ drift):

```
  // Bỏ 100 mẫu đầu (chờ cảm biến ổn định)
  for (int i = 0; i < 100; i++) {
    mpu.getMotion6(&ax0, &ay0, &az0, &gx0, &gy0, &gz0);
    delay(2);
  }
  // Lấy trung bình 1000 mẫu để tìm offset
  long sumGz = 0;
  for (int i = 0; i < 1000; i++) {
    mpu.getMotion6(&ax0, &ay0, &az0, &gx0, &gy0, &gz0);
    sumGz += gz0;
    delay(2);
  }
  gyroZOffset = (float)sumGz / 1000.0;  // Giá trị trung bình khi ĐỨNG YÊN
```

> **Tại sao cần calibrate?** Gyro luôn có giá trị nhỏ khác 0 khi đứng yên (bias). Nếu không trừ offset, góc yaw sẽ trôi liên tục (drift).

3.2 `IMU_read()` — Đọc và tích phân góc Yaw

```
void IMU_read() {
  int16_t ax, ay, az, gx, gy, gz;
  mpu.getMotion6(&ax, &ay, &az, &gx, &gy, &gz);  // Đọc 6 trục
```

```
  unsigned long now = micros();
  float dt = (now - imu_prevTime) / 1000000.0;  // Tính dt (giây)
  imu_prevTime = now;
  if (dt <= 0 || dt > 0.5) dt = 0.02;  // Bảo vệ giá trị bất thường
```

Tính góc xoay:

```
  float gyroZ = ((float)gz - gyroZOffset) / 131.0;  // Raw → °/s (trừ offset)
  if (fabs(gyroZ) < YAW_DEAD_ZONE) gyroZ = 0.0;     // Lọc nhiễu nhỏ < 0.4°/s
  yaw += gyroZ * dt;  // Tích phân: vận tốc góc × thời gian = góc
```

> **Công thức:** `yaw(t) = yaw(t-1) + ω_z × Δt` Dead zone 0.4°/s giúp yaw không bị trôi khi robot đứng yên do nhiễu cảm biến.

Chuẩn hóa góc:

```
  if (yaw > 180.0)  yaw -= 360.0;   // Giữ yaw trong [-180°, 180°]
  if (yaw < -180.0) yaw += 360.0;
}
```

3.3 `IMU_reset()` & `getYaw()`

```
void IMU_reset() { yaw = 0.0; }   // Reset yaw về 0 (dùng khi chuyển phase)
float getYaw() { return yaw; }     // Getter lấy giá trị yaw hiện tại
```

PHASE 4: MOTOR (Motor.ino)

4.1 `Motor(LPWM, RPWM)` — Điều khiển 2 động cơ

```
void Motor(int LPWM, int RPWM) {
  digitalWrite(STBY, 1);                    // Đảm bảo driver đang bật
  LPWM = constrain(LPWM, -255, 255);       // Giới hạn PWM trái
  RPWM = constrain(RPWM, -255, 255);       // Giới hạn PWM phải
```

Xác định chiều quay Motor trái:

```
  if (LPWM >= 0) {
    digitalWrite(IN1, HIGH);  // Quay tiến
    digitalWrite(IN2, LOW);
  } else {
    digitalWrite(IN1, LOW);   // Quay lùi
```

```
    digitalWrite(IN2, HIGH);
  }
```

Xác định chiều quay Motor phải:

```
  if (RPWM >= 0) {
    digitalWrite(IN3, LOW);   // Quay tiến
    digitalWrite(IN4, HIGH);
  } else {
    digitalWrite(IN3, HIGH);  // Quay lùi
    digitalWrite(IN4, LOW);
  }
```

Cấp nguồn PWM (điều tốc):

```
  analogWrite(ENA, abs(LPWM));  // Cấp PWM trái (0-255)
  analogWrite(ENB, abs(RPWM));  // Cấp PWM phải (0-255)
}
```

> `abs()` vì chiều quay đã được set bởi IN1-IN4, PWM chỉ quyết định TỐC ĐỘ.

> **Tiếp tục:** Xem Phần 2 - Odometry, PID, WiFi, Bluetooth